Accelerating Cognitive Sequential Recommenders with Approximate Nearest Neighbor Retrieval for Music Recommendation

Jason Tobin

Computer Science, San Jose State University

CS 274

jason.tobin@sjsu.edu

Abstract

Sequential recommendation in music streaming must capture both short-term session context and long-term listening habits. Presented here is MemSeqRec, a model that fuses cognitive memory principles from ACT-R theory with a SASRec-style Transformer and an approximate-nearest-neighbour (ANN) re-ranking stage. The retrieval stage combines ACT-R-derived base-level learning (BLL) decay weights and spreading-activation context with self-attentive sequence modelling. The fine-ranking stage re-scores k=1500 ANN candidates using a learned re-ranker trained with a sampled softmax pool loss over 64 in-batch negatives. This is evaluated on the Deezer music-streaming dataset (7,063 users, 50,000 tracks) against the ACTR-BPR collaborative filtering baseline. Despite architectural sophistication, MemSeqRec achieves NDCG@10 = 0.00445 against ACTR-BPR's NDCG@10 = 0.0855, a gap analyzed in depth. The findings highlight fundamental challenges in combining decay heuristics with end-to-end gradient learning.

Introduction

Music recommendation is an incredibly impactful recommender system that most people experience throughout their daily life. With this, comes a humongous issue on the scale of data and users. Spotify's user base consists roughly 751 million users [4], an uncomputable number for recommender systems to realistically do in real time. Unlike movie recommendations, music is not only genre specific, but activity specific. This means that sequential recommender systems not only have a lot of song data to take into account, but a lot of context based on each individual user. Furthermore, music catalogs are also mutable, allowing new or existing artists to upload new music which needs to be recommended as well.

Purely session based models typically discard the historic context in which a user is listening in. The AU2ACTR framework bridges the gap between historical and current context by encoding activation scores from a user's history. This is done through the use of the ACT-R cognitive-based architecture inside a recommendation pipeline. Inside this framework, ACT-R is the reference baseline as it constructs a user representation by weighting past items with ACT-R BLL decay scores and optimises a Bayesian Personalised Ranking. Each recommended track was compared against all existing tracks in their dataset to determine similarity, and the cognitive embedding, referred to as ACT-R, took a large amount of time per listening session. This means that their complexity was far too high. Although this does work to an extent it limits the possible results that can be recommended to a user.

In this paper, is a newly proposed network architecture, MemSeqRec, which builds upon the ACT-R feature extraction with an SASRec-style Transformer recommendation system for sequential modelling and a two-stage retrieval pipeline using an Approximate Nearest Neighbor module for candidate generation followed by a re-ranker.

I. Related Work

A. Session-Based and Sequential Recommendation

Session-based methods such as GRU4Rec [9] and SASRec [2] model the change of user interest within or across sessions using recurrent or self-attentive architectures. These models operate

purely on item reoccurrence and ignore any temporal-based signals that a user may give.

B. Cognitive Memory Models in Recommendation

ACT-R's base-level learning (BLL) equation formalises how memory activation decays as a power function of time and usage. Several recommender systems use this decay function to weigh historical interactions. The AU2ACTR framework implements spreading activation across a session graph and the BLL weights.

II. Dataset

The dataset used for this project comes from the Deezer-RecSys25 collection. It is a real-world streaming dataset, produced by Deezer. The raw data consists of sharded Parquet files which contain user session streams, and pre-computer track embeddings. These track embeddings are provided outright by Deezer as a method of saving on future computation time as well as serve as a way to anonymize tracks without infringing on copyright. In this project, there is a preprocessing script, which performs two sequential passes to filter the data and not load all of it into memory.

The dataset consists of over 3,000,000 user listening sessions and over 900,000,000 song embeddings. In order to filter these to usable sessions, there is a minimum-session filter applied which requires over 300 distinct sessions per user [2]. This leads the total working dataset to 7,063 users, and 50,000 tracks. Each stream of a track contains the user_id, session_id, track_id, and a Unix timestamp. Then, for each listening session, they are sorted chronologically within each user's session, which gives a full and chronological listening history. The reason that 300 sessions is required is for three reasons. Computation time upon training, context history for the ACT-R modules (emulated or not), as well as making sure that user song choices are organically chosen. One issue with sequential recommendation in the music sense, is that algorithms, perfect or not, already exist to provide music to a user. The data must reflect organic listening sessions rather than algorithmic listening sessions. Increasing how many sessions are included increases the amount of organic sessions included for each user.

Furthermore, each track has two pre-computed embedding vectors provided by the kind people at Deezer. These are SVD embeddings, derived from user-track cooccurance, as well as audio embeddings. The audio embeddings are, in themselves, a content-based feature vector, and save the characteristics of the track without, as previously mentioned, infringing on copyright.

Continuing, the data is split into train, validation, and test sets, with the split taking a per-user methodology rather than set percentage. For this project, it is based on the work done by Tran et. al., [2], where for any given user, all listening sessions except the last 20 are used for the training set. Then the last 20 are split into 10 and 10, with one group of 10 used for validation, and the other used for training. This guarantees that the test and validation are specifically built on future recommendation rather than per-user overfitting.

Table I. Dataset Statistics

Statistic	Value
Users	7,063
Tracks	50,000
Min. sessions / user	300
Sequence length L	30
Session step	20
Val. sessions / user	10
Test sessions / user	10

III. Methodology

A. Overview

There are 4 components to my proposed MemSeqRec architecture. An ACT-R feature extractor, SASRec-style encoder, a gated fusion layer, and an ANN with re-ranker.

The pipeline is as follows:

ACT-R Weights to Session Aggregation

- Element by Element SASRec Encoding
- Gated Fusion
- ANN Retrievel (k = 1500)
- Re-ranking
- Final ranking

B. ACT-R Feature Extraction

For each user session, following from the architecture outlined in [2], the BLL activation scores from each song must be used for every

possible candidate item. Within a user's recent listening graph these weights are computing to form a cognitive context vector containing the importance of each song per user.

C. SASRec-Style Encoder

Continuing, the last (L=30) interacted with tracks are embedded and passed into a self-attention block with 2 heads. The output is the hidden state of the final sequence position of each track. Essentially the positional encodings of each track are learnable, and therefore trained to understand.

D. Gated Fusion

Very simply, we do not want the model to only use ACT-R based sequential recommendation. That is done in [2] and is useless to completely rely on for our purposes. Therefore the model takes into account only a learned amount of the ACT-R weights vs the SASRec-style transformer. This is then entered in a fusion layer, with the following:

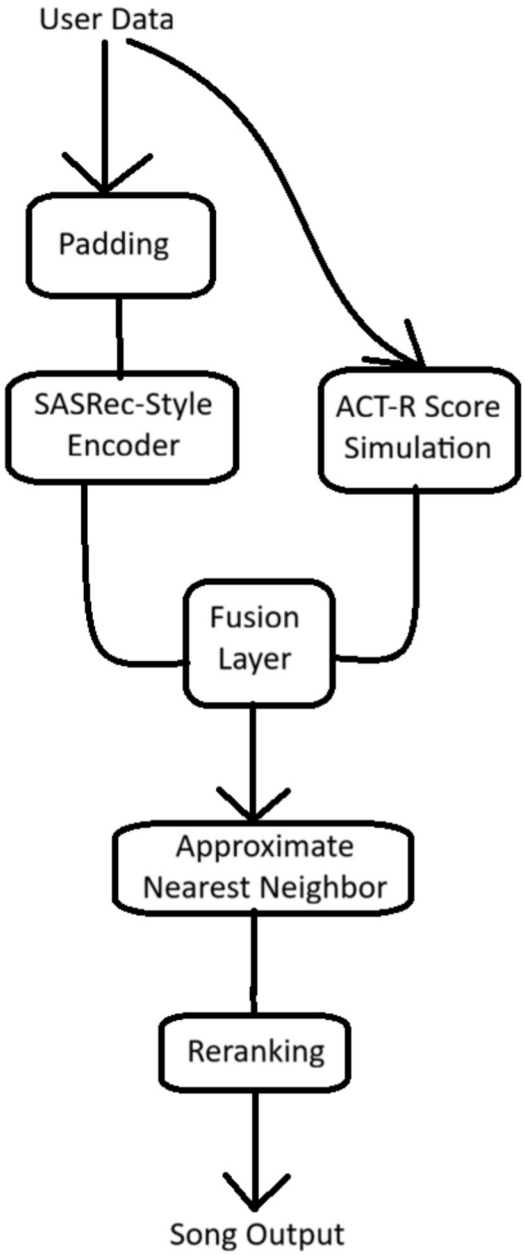
$$g = \sigma(W\_g [h\_actr ; h\_seq] + b\_g)$$

$$h = g \odot h\_actr + (1-g) \odot h\_seq$$

h_actr is a weighted average of a user's favorite items.
h_seq is the short-term sequential pattern from what a user has just listened to, built by the SASRec transformer. The two are combined and concatenated to form one vector, giving a scaler per dimension.

E. Approximate Nearest Neighbors

The ANN portion of this is very basic, using the standard implementation. This is necessary as the whole dataset doesn't need to be sorted fully nor does the model output in the same order that it has been entered in.



IV. Experiments

A. Baseline and Evaluation

For this project, the baseline is computed via the architecture outlined in [2]. As a reminder the main aim of this project is to improve inference time while maintaining accuracy. Therefore all comparisons will be made against reproduced results of the architecture provided by Tran et. al.

This base line is the ACT-R item embeddings without a SASRec-Style transformer.

The models are all evaluated with the same protocols, using the same random test users, of seeds 1013, 2791, 4357, 6199, 7907. For each test session, the model ranks the next item from the full catalogue for the entire 50,000 track dataset.

The model is then measured on:

A. NDCG@10: Normalised Discounted Cumulative Gain at cut-off 10
   a. This stands as the primary metric for measuring model performance.
B. Recall@10: Fraction of held-out items appearing in the top-10
C. Repr@10: Representation score measuring diversity relative to a test user's listening history
D. NDCG_exp@10: NDCG but it is decomposed over exploration (new tracks) vs. repeated items (found in user history).
E. Pop@10: Mean popularity rank of top-10 items.
   a. Serves as a method of comparing recommendation methods vs popularity of songs to a user.

B. Hyperparameters of MemSeqReq

Table II.

Hyperparameters of MemSeqRec

Parameter	Value
Embedding dimension d	128
Transformer blocks B	2
Attention heads H	2
Sequence length L	30
Favourite items N_f	50
ANN pool size k	1,500
Re-ranker hidden dim	256
Dropout rate	0.10
λ_task	0.60
L2 emb. regularisation	1×10^{-5}
Peak learning rate η_max	1×10^{-3}
Min. learning rate η_min	1×10^{-8}
Warm-up steps	221
Total training steps	22,100
Batch size	512
Epochs	100

V. Results

A. Main Comparison

The following table (Table III) reports the test-set results.

Table III. Test-Set Results on Deezer, best is designated with ★

Metric	ACT-R	MemSeqRec
NDCG@10	0.0855 ★	0.00445 ± 0.00008
Recall@10	0.0804 ★	0.00384 ± 0.00007
Repr@10	0.716 ★	0.218 ± 0.002
NDCG_rep@10	—	0.00455 ± 0.00010
NDCG_exp@10	0.0193 ★	0.00143 ± 0.00012
Recall_rep@10	—	0.00420 ± 0.00012
Recall_exp@10	—	0.00222 ± 0.00015
Pop@10	—	0.218 ± 0.000
Best epoch	92	38
Best val. loss	—	0.0888

The following table (Table III) reports the test-set results. Across all primary metrics it is evident that MemSeqRec is insufficient and achieves nowhere near the level that base ACT-R does. MemSeqRec obtains a NDCG@10 = 0.00445 versus ACT-R of 0.0855, reaching a 19x in the primary metric. This will be discussed further in the conclusions.

B. Training and Reasoning

After the poor performance across the same dataset, it is important to examine and locate outstanding issues that the model architecture faces.

Table IV. Selected MemSeqRec Validation Metrics Across Training

Epoch	LR	Val-Loss	Val NDCG@10
0	1.00×10^{-3}	0.3115	—
9	9.8×10^{-4}	0.1225	0.00271
19	9.1×10^{-4}	0.1024	0.00577 ★
29	8.0×10^{-4}	0.0929	0.00476
38	6.8×10^{-4}	0.0888 ★	—
49	5.1×10^{-4}	0.0973	0.00233
69	2.1×10^{-4}	0.0940	0.00422
99	≈0	0.0933	0.00197

Evidently, it can be seen that the model performs best after 19 epochs, and deteriorates from that point onward.

C. Inference Times

Finally, the goal of the project was to create a similarly accurate, but faster (in inference time) model. Although so far findings do not show the accuracy held up, it is possible that the inference time did increase.

Table V. Inference Times of ACT-R and MemSeqRec

Model	Total (s)	Mean Batch (ms)	Std (ms)	Per-user (ms)
ACT-R	57.09	2038.9	101.9	20.39
MSR	47.66	1702.1	80.6	17.02

Although the accuracy of the MemSeqRec model is lackluster, it can be seen that even with the two added modules of the fusion layer, as well as the approximate nearest neighbors layer, that the model's inference times was lower by ~16%.

VI. Discussion and Analysis

The goal of this project was fairly simple in concept, but difficult in execution. Sequential recommendation, even on a small scale, is extremely difficult and can lead to an incredible amount of issues. It is seen in this project, that attempting to improve existing solutions, with a large amount of approximation, has led to an extreme loss in accuracy. The loss in accuracy itself is not unexpected, but the size of it is. Although the validation loss, throughout training, saw continuous improvement, the ranking metrics do not follow the same results. The disconnect is most likely due to the easy task of ranking inside 50,000 tracks, versus the full catalog of 900,000,000. The model learns to separate inside this limited subset, but cannot distinguish once the magnitude of the dataset increases substantially. *The author of this paper would like to note that the full dataset was tested, however small amounts of epochs (< 3) would take over 48 hours to train, and was impossible to realistically train in a meaningful way.

One of the reasons it is believed that the model accuracy in almost all metrics was so low is due to the bottleneck at the ANN level. The size of the pool (k = 500, 1500), is merely 3% of the catalog. Therefore the re-ranker does not find itself with the task of ranking over the full catalog of songs. This majorly helps the inference time, and is believed to be the main source of the inference time performance increase. This means that, from a purely probability standpoint, it is unlikely for a given track to be the next track a user may choose to listen to.

Furthermore, in comparison it is seen that the ACT-R methodology optimizes directly over the full item set and is a primary measurement for track sequential recommendation. Whereas the architecture seen in MemSeqRec only denotes this to be a part of the decision making process. As found in [3], it is evident that humans like to repeat patterns and, given the task of music recommendation, listen to the same tracks multiple times. Therefore, if our model was to just recommend the same tracks a user has already listened to, then it would most likely achieve an even greater accuracy. This can be seen in the Repr@10 measurement of 0.218 for MemSeqRec, versus that of 0.716 for the ACT-R baseline. This means that the proposed new architecture does far worse at representing a user's listening history. This was also one of the goals of the project as well, as it is one of the limitations that is provided by the ACT-R architecture. Exploration is achieved with this model, even if the exploration metric is not that good. This also means that the fusion layer is not one which accurately values the ACT-R as much as it should. This may require a rebuild from the ground up on how the model learns the rate at which to intake the user listening history.

Although the model does underperform the baseline overall, there were two desirable properties of this architecture. First of all, since the ACT-R provided method is primarily based on user listening rather than content based implementations, this model has a far greater use of exploration, and does look outside the user-history. This is important for long-tail discovery, a formidable issue in music sequential recommendation.

Finally, there are a few main areas in which this could be improved but there was not enough time to improve this. One of the biggest improvements that could be made is through Gate Regularization. Adding a regularization term to the gate to prevent early saturation on the value of exploration vs familiar tracks may allow the model to actually learn which to favor. Furthermore, this could even be done without regularization but through initialization of a 50/50 split across old and new tracks. Furthermore, a longer warmup and lower peak learning rate could be used. In the current implementation a cosine decay is used for the learning rate which could lead to a lack of substantial learning and the model could be limited.

One way that the model could be massively improved is through the implementation of a pretrained model. It is seen in [2] that a pretrained model leads to even further gains in the performance of a model. Incorporating this, alongside the fusion layer, may actually make exploration as well as familial recommendation possible.

Finally, the final area in which I believe this could be improved, although I'm sure there are more, is in the utilization of the audio feature vectors provided by Deezer, as well as the SVD embeddings. This could be used to compute track similarity based on the audio features, rather than just user listening history and base embedding similarity. This was originally omitted due to the possible increase in inference time, however it is showcased through this architecture that inference time is actually quite sped up in sequential recommendation scores. Therefore, it is not out of the realm of possibility to say that it may be possible to implement this without destroying the models performance time.

VII. Conclusion

In this paper, we have demonstrated the ability for a model to increase the inference time with a substantial lack of accuracy. Although the goals of this architecture was to increase inference time (achieved) and maintain accuracy levels, it is shown that the amount of approximation that is implemented in this methodology is far too extreme to maintain our goals.

This is seen in the MemSeqRec architecture, a cognitive-attentive sequential recommendation system that integrates the ACT-R memory decay as well as a SASRec-style transformer, with ANN and re-ranking in order to sequentially recommend the next listened to tracks by a user. Although the model did successfully train, it seems that the total dataset size, along with the model architecture caused it to stagnate in the performance that we achieved.

The proposed model is found to underperform by a substantial margin against the Deezer Benchmark that is provided in [2].

With all of this in mind it is still important to realize that the findings in this paper do add to important discoveries in the sequential recommendation platform. In this paper, and architecture, it is found that through addition of the approximations provided, exploration of tracks is increased, a goal of the project overall.

Future work needs to be done in order to address the poor performance of the model itself. This includes a total re-implementation of the gated fusion layer, a redesign of the learning rate methodology, utilization of the audio feature vector provided by Deezer, as well as stronger hardware in order to fully utilize the provided dataset.

We have identified that one of the biggest areas which could improve the performance of the model is through the use of pretraining a sub-model, only on the ACT-R activation alone, and using the methodology employed here to improve the exploration of similar architectures.

Acknowledgment

I would like to officially thank my good friend Riley Dinklebach for allowing me to remotely use his hardware for this project specifically as without him the findings and experiments I ran would not have been anywhere near comprehensive. Before using his hardware, I was using my GTX 1080ti and a single model would take >48 hours to train on a successful run.

References

[1] Giovanni Gabbolini and Derek Bridge. 2021. Play It Again, Sam! Recommending Familiar Music in Fresh Ways. In Proceedings of the 15th ACM Conference on Recommender Systems (RecSys '21). Association for Computing Machinery, New York, NY, USA, 697–701. https://doi.org/10.1145/3460231.3478866

[2] Viet Anh Tran, Bruno Sguerra, Gabriel Meseguer-Brocal, Lea Briand, and Manuel Moussallam. 2025. "Beyond the past": Leveraging Audio and Human Memory for Sequential Music Recommendation. In Proceedings of the Nineteenth ACM Conference on Recommender Systems (RecSys '25). Association for Computing Machinery, New York, NY, USA, 509–514. https://doi.org/10.1145/3705328.3748018

[3] Davide Abbattista, Vito Walter Anelli, Tommaso Di Noia, Craig Macdonald, and Aleksandr Vladimirovich Petrov. 2024. Enhancing Sequential Music Recommendation with Personalized Popularity Awareness. In Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24). Association for Computing Machinery, New York, NY, USA, 1168–1173. https://doi.org/10.1145/3640457.3691719

[4] *About Spotify* https://newsroom.spotify.com/company-info/

[5] Taatgen, N.A. & Anderson, J.R. (2008). ACT-R. In R. Sun (ed.), Constraints in Cognitive Architectures. Cambridge University Press, pp 170-185.

[6] Yizhou Dang, Yuting Liu, Enneng Yang, Guibing Guo, Linying Jiang, Xingwei Wang, and Jianzhe Zhao. 2024. Repeated Padding for Sequential Recommendation. In Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24). Association for Computing Machinery, New York, NY, USA, 497–506. https://doi.org/”10.1145/3640457.3688110

[7] Gaowei Zhang, Yupeng Hou, Hongyu Lu, Yu Chen, Wayne Xin Zhao, and Ji-Rong Wen. 2024. Scaling Law of Large Sequential Recommendation Models. In Proceedings of the 18th ACM Conference on Recommender Systems (RecSys '24). Association for Computing Machinery, New York, NY, USA, 444–453. https://doi.org/10.1145/3640457.3688129

[8] Kang, W.-C., & McAuley, J. (2018). Self-Attentive Sequential Recommendation (Version 1). arXiv. https://doi.org/10.48550/ARXIV.1808.09781

[9] B. Hidasi et al., 'Session-Based Recommendations with Recurrent Neural Networks,' Proc. ICLR, 2016.

[10] Dataset: Deezer-RecSys25 organic listening events dataset, https://zenodo.org/records/17154089